Course: Deep Learning

Submitted to: Ma’am Nabeela Bibi

Name: Abdul Wahab Babar (01-134232-210)

M.Asher (01-134232-228)

Class: BSCS (7)

DEPARTMENT OF COMPUTER SCIENCE

BAHRIA UNIVERSITY | ISLAMABAD H11- CAMPUS

Abstract

This project reproduces a handwritten text recognition workflow using TrOCR, a transformer-based optical character recognition model. The pretrained checkpoint microsoft/trocr-small-stage1 was fine-tuned on the IAM handwritten line dataset using a writer-independent training, validation, and test split. The usable split contained 5,419 training, 1,539 validation, and 1,480 test samples. The model was trained for three epochs in Google Colab with a GPU. Performance was evaluated using Character Error Rate (CER) and Word Error Rate (WER). On the held-out test set, the fine-tuned model achieved 7.75% CER and 15.11% WER. The results show a clear improvement over the pretrained baseline on the validation set, while qualitative examples reveal remaining errors involving proper names, numbers, and visually ambiguous words.

1. Introduction

Optical Character Recognition (OCR) converts text in images into machine-readable text. Handwritten text recognition is particularly challenging because handwriting varies between writers and may contain connected characters, irregular spacing, different slants, and inconsistent letter shapes. A recognition system must learn both visual patterns and the sequence structure of language.

TrOCR uses a Transformer encoder-decoder architecture for OCR. An image encoder extracts visual representations from the input image, and a text decoder generates the corresponding character or subword sequence. The selected small pretrained checkpoint provides an initial model that can be adapted to handwritten line images through fine-tuning.

2. Research Paper Overview

The selected research work is “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models” by Li and colleagues. Its central approach is to formulate OCR as an image-to-text sequence generation task and to use pretrained Transformer components rather than relying only on traditional character-level classification pipelines.

For this reproduction, the microsoft/trocr-small-stage1 checkpoint was selected as the starting model. The experiment focuses on adapting this pretrained model to handwritten IAM line images and measuring its recognition performance. The implementation is a practical reproduction on the available dataset and hardware; it should not be interpreted as an exact replication of every experiment or result reported in the original paper.

3. Dataset

The experiment used the IAM Handwriting Database line-recognition data. The dataset contains handwritten text-line images paired with their transcriptions. The data files were obtained from the Zenodo record used for this project. Only annotation records marked with status “ok” were retained, and the official writer-independent split identifiers were used to separate training, validation, and testing data.

Subset	Usable samples	Purpose
Training	5,419	Model fine-tuning
Validation	1,539	Monitoring and model selection
Testing	1,480	Final held-out evaluation
Total	**8,438**	**Usable samples across official splits**

The annotation file contained 13,353 records overall, including records not marked as usable. The original official split identifier counts were larger than the usable counts because records with non-usable status were excluded. The final counts above therefore refer to filtered, usable samples, not to the total number of annotation records.

Images were processed at a target size of 384 × 384 pixels. Transcriptions were tokenized with a maximum target length of 128 tokens. The longest observed transcription contained 23 tokens, and no sample exceeded the configured maximum.

4. Methodology and Implementation

4.1 Data preparation

The annotation records were parsed and matched with their corresponding line images. The data was filtered to retain valid records and assigned to the writer-independent train, validation, and test partitions. Dataset samples were visualized to check that image paths and transcriptions were paired correctly. Image tensors were converted to floating-point format and labels were tokenized for the TrOCR processor.

4.2 Model and configuration

The pretrained model was loaded from microsoft/trocr-small-stage1. During implementation, the decoder start-token configuration was checked and corrected to match the checkpoint's expected special-token IDs: decoder_start_token_id = 2, pad_token_id = 1, and eos_token_id = 2. This was important because an incorrect decoder start token caused generation to behave incorrectly. The corrected configuration was applied consistently to the model and generation configuration.

4.3 Fine-tuning setup

Setting	Value
Pretrained checkpoint	microsoft/trocr-small-stage1
Epochs	3
Learning rate	5×10^{-5}
Per-device batch size	2
Gradient accumulation steps	4
Weight decay	0.01
Optimizer	AdamW
Precision	FP16 when CUDA was available
Random seed	42
Hardware	Google Colab GPU (Tesla T4)

The model was evaluated at the end of each epoch, and the best checkpoint was selected using validation CER. Training completed for three epochs, corresponding to 2,034 optimizer steps. The final trained model and experiment artifacts were saved to Google Drive.

5. Evaluation Metrics

Character Error Rate (CER) measures the number of character-level insertions, deletions, and substitutions relative to the number of characters in the reference transcription. Word Error Rate (WER) measures the equivalent word-level errors relative to the number of reference words. Lower values indicate better recognition performance. CER is useful for character-level transcription quality, while WER gives a more direct indication of complete-word recognition errors.

6. Results

The pretrained baseline was evaluated on the validation set before fine-tuning. The fine-tuned model was evaluated on both the validation set and the held-out test set.

Model	Dataset	CER	WER
Pretrained baseline	Validation	13.20%	35.24%
Fine-tuned TrOCR	Validation	7.07%	14.46%
Fine-tuned TrOCR	Test	7.75%	15.11%

On validation data, fine-tuning reduced CER from 13.20% to 7.07% and WER from 35.24% to 14.46%. This corresponds to an absolute reduction of approximately 6.13 percentage points in CER and 20.78 percentage points in WER. The held-out test results (7.75% CER and 15.11% WER) are slightly higher than validation results, which is expected because the test set was not used for model selection. These results indicate that fine-tuning improved recognition quality on this dataset.

The repository includes the training history, final metrics, baseline results, and test predictions. The plots folder contains dataset sample images, training loss curves, validation metrics, and a baseline-versus-fine-tuned comparison.

7. Qualitative Analysis

Visual inspection of predictions showed that the model can recover many words and sentence structures correctly. However, errors remained in proper names, uncommon words, numbers, and phrases whose handwriting is visually ambiguous. For example, a person's age could be confused with another number, and names such as place names or composer names were sometimes transcribed incorrectly. Small character differences can also change the meaning of a word.

These examples show why aggregate metrics and manual inspection should be used together. CER and WER summarize overall error rates, while individual predictions help identify the kinds of mistakes that remain.

8. Limitations and Possible Improvements

- The experiment used a small pretrained checkpoint and a limited three-epoch fine-tuning schedule.
- The usable IAM split contains fewer samples than the full annotation inventory because non-usable records were filtered out.
- Recognition errors remain for proper nouns, rare vocabulary, and similar-looking handwritten characters.
- Results may vary with random initialization, library versions, GPU availability, and training settings.
- Further work could explore longer training with careful validation monitoring, data augmentation, hyperparameter tuning, and error analysis by writer or text category.

9. Conclusion

This assignment implemented a TrOCR fine-tuning pipeline for handwritten line recognition using the IAM dataset. The data was prepared using writer-independent partitions, the pretrained microsoft/trocr-small-stage1 checkpoint was configured correctly, and the model was fine-tuned for three epochs on a Google Colab GPU. The fine-tuned model achieved 7.07% validation CER and 14.46% validation WER, followed by 7.75% test CER and 15.11% test WER. Compared with the pretrained validation baseline, the fine-tuned model showed a substantial improvement. Remaining errors suggest opportunities for additional training and more targeted handling of names and numbers.

10. Project Repository and Reproducibility

The GitHub repository contains the implementation source files, completed notebook, dependency list, experiment plots, metrics, training history, and test predictions. The trained model weights and original dataset are stored separately in Google Drive and are not included in the GitHub repository because of their size and dataset distribution considerations.

GitHub repository URL: https://github.com/abdulwahab-0/TrOCR-Handwritten-Text-Recognition/tree/main

Main implementation files: src/model.py, src/dataset_preparation.py, src/training.py, and src/evaluation.py.

Notebook: notebooks/TrOCR_Assignment_02.ipynb

References

1. Li, M., et al. (2021). TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models. arXiv:2109.10282. https://arxiv.org/abs/2109.10282

2. IAM Handwriting Database. Dataset and documentation. https://www.iam.unibe.ch/fki/databases/iam-handwriting-database
3. Microsoft. microsoft/trocr-small-stage1 model card. Hugging Face Model Hub. https://huggingface.co/microsoft/trocr-small-stage1
4. Zenodo. IAM line-recognition data record used in this project. https://zenodo.org/records/19003721

Dataset note: Users should consult the IAM dataset's official terms and the linked Zenodo record before redistributing data. This repository provides code and derived experiment outputs rather than republishing the original dataset.